Manual Técnico: HAILCAST 1D

Richard Perdomo Arcila

10 de febrero de 2026

Índice

1. Módulo: `__init__.py` 3
- 1.1. Propósito . . . 3
- 1.2. Arquitectura de la API . . . 3
- 1.3. Funciones y clases exportadas . . . 3
  - 1.3.1. Simulación determinista . . . 3
  - 1.3.2. Simulación estocástica . . . 3
  - 1.3.3. Modelo atmosférico . . . 3
  - 1.3.4. Visualización . . . 4
  - 1.3.5. Análisis . . . 4
- 1.4. Definición de la interfaz pública . . . 4
- 1.5. Rol dentro del flujo del modelo . . . 4

2. Módulo: `cloud_model` 4
- 2.1. Propósito . . . 4
- 2.2. Clase `CloudProfile` . . . 5
  - 2.2.1. Inicialización . . . 5
  - 2.2.2. Perfiles predefinidos . . . 5
- 2.3. Campos físicos definidos . . . 5
  - 2.3.1. Contenido de agua líquida . . . 5
  - 2.3.2. Velocidad vertical . . . 6
  - 2.3.3. Temperatura ambiental . . . 6
  - 2.3.4. Densidad del aire . . . 6
- 2.4. Rol dentro del modelo . . . 7
- 2.5. Limitaciones . . . 7

3. Módulo: `hail_growth.py` 7
- 3.1. Propósito . . . 7
- 3.2. Función principal: `simulate_trajectory` . . . 8
- 3.3. Modelo físico . . . 8
- 3.4. Integración numérica . . . 9
- 3.5. Registro de resultados . . . 9
- 3.6. Consideraciones técnicas . . . 10

4. Módulo: `hail_growth_stochastic.py` 10
4.1. Propósito . . . 10
4.2. Funciones principales . . . 10
4.2.1. `generate_initial_conditions_mc` . . . 10
4.2.2. `run_montecarlo` . . . 11
4.3. Consideraciones técnicas . . . 11

5. Módulo: `hailcast1d.py` 12
5.1. Propósito . . . 12
5.2. Funciones principales . . . 12
5.2.1. 1. `run_deterministic_simulation` . . . 12
5.2.2. 2. `run_montecarlo_simulations` . . . 13
5.2.3. 3. `run_case` . . . 13
5.2.4. 4. `run_updraft_set` . . . 13
5.2.5. 5. `run_all_cases` . . . 13
5.3. Consideraciones técnicas . . . 14

6. Módulo: `plot_hail_trajectories.py` 14
6.1. Propósito . . . 14
6.2. Funciones principales . . . 14
6.2.1. `plot_hailstone_trajectories` . . . 14
6.3. Consideraciones tecnicas . . . 15

1. Módulo: `__init__.py`

1.1. Propósito

El archivo `__init__.py` define la API pública del paquete `hailcast1d`. Su función es exponer las funciones y clases principales del modelo sin que el usuario necesite conocer la estructura interna de los módulos.

Permite importar directamente desde el paquete:

```
from hailcast1d import run_all_cases, CloudProfile
```

en lugar de realizar importaciones desde submódulos internos.

1.2. Arquitectura de la API

El módulo actúa como capa de acceso y organiza los siguientes componentes:

- Simulación determinista
- Simulación estocástica
- Modelo de nube
- Visualización
- Análisis de resultados

1.3. Funciones y clases exportadas

El archivo importa y expone los siguientes elementos:

1.3.1. Simulación determinista

- `run_deterministic_simulation`
- `run_case`
- `run_updraft_set`
- `run_all_cases`

Origen: `hailcast1d.py`

1.3.2. Simulación estocástica

- `run_montecarlo`

Origen: `hail_growth_stochastic.py`

1.3.3. Modelo atmosférico

- `CloudProfile`

Origen: `cloud_model.py`

1.3.4. Visualización

- `animate_hailstone`
- `animate_hail_growth`

Origen: `hail_animation.py`

1.3.5. Análisis

- `analyze_all_profiles`

Origen: `hail_analysis.py`

1.4. Definición de la interfaz pública

La variable especial:

```
__all__ = [...]
```

define explícitamente qué símbolos forman parte de la API pública cuando se utiliza:

```
from hailcast1d import *
```

Esto garantiza:

- Control sobre la interfaz externa
- Encapsulamiento de módulos internos
- Estabilidad de la API frente a refactorizaciones internas

1.5. Rol dentro del flujo del modelo

El archivo `__init__.py` no implementa física ni algoritmos numéricos. Su función es estructural:

1. El usuario importa el paquete.
2. El paquete expone la API definida.
3. Las funciones redirigen hacia los módulos internos.

De esta manera, el paquete adquiere una estructura modular, mantenible y escalable.

2. Módulo: `cloud_model`

2.1. Propósito

El módulo `cloud_model` define el perfil vertical estacionario de la nube utilizado por el modelo HAILCAST 1D. Proporciona las variables ambientales necesarias para la simulación del crecimiento y dinámica del granizo:

- Contenido de agua líquida (LWC)

- Velocidad vertical del aire
- Temperatura ambiental
- Densidad del aire

El modelo asume una nube estacionaria con perfiles verticales prescritos.

2.2. Clase `CloudProfile`

2.2.1. Inicialización

```
CloudProfile(updraft_type="strong", lwc_peak=None, w_peak=None,
             z_min=0.0, z_max=10000.0, z_center=4000.0)
```

Parámetros

Parámetro	Descripción
`updraft_type`	Tipo de corriente: `"strong"`, `"dry"`, `"weak"`
`lwc_peak`	Contenido máximo de agua líquida (kg/m^3)
`w_peak`	Velocidad máxima ascendente (m/s)
`z_min`, `z_max`	Límites verticales del dominio (m)
`z_center`	Altura del máximo ascenso (m)

2.2.2. Perfiles predefinidos

Si no se especifican valores manuales, se adoptan los siguientes valores por defecto:

Tipo	w_{peak} (m/s)	LWC_{peak} (kg/m^3)
strong	15.0	$1{,}2 \times 10^{-3}$
dry	8.0	$0{,}6 \times 10^{-3}$
weak	3.0	$0{,}3 \times 10^{-3}$

2.3. Campos físicos definidos

2.3.1. Contenido de agua líquida

Se modela mediante un perfil gaussiano centrado en z_{center}:

$$LWC(z) = LWC_{peak} \exp\left(-\frac{(z - z_{center})^2}{2\sigma^2}\right) \tag{1}$$

con $\sigma = 1500$ m.
Se impone la condición:

$$LWC(z) \geq 0 \tag{2}$$

2.3.2. Velocidad vertical

Se define un perfil gaussiano:

$$w(z) = w_{peak} \exp\left(-\frac{(z - z_{center})^2}{2\sigma^2}\right) \tag{3}$$

con $\sigma = 2000$ m.
Se incluyen regiones de recirculación:

- Parte superior: $w = -5$ m/s
- Parte inferior: $w = -2$ m/s

Esto permite modelar ciclos de ascenso y descenso del granizo dentro de la nube.

2.3.3. Temperatura ambiental

Se adopta un gradiente térmico estándar:

$$T(z) = T_{surface} - \Gamma z \tag{4}$$

donde:

- $\Gamma = 0{,}0065$ K/m
- $T_{surface} = 288{,}15$ K

Se impone un límite inferior:

$$T(z) \geq 180 \text{ K} \tag{5}$$

2.3.4. Densidad del aire

Se utiliza una aproximación barométrica:

$$p(z) = p_0 \left(1 - \frac{\Gamma z}{T_0}\right)^{5{,}256} \tag{6}$$

y la ecuación de estado del gas ideal:

$$\rho(z) = \frac{p(z)}{RT(z)} \tag{7}$$

donde:

- $p_0 = 101325$ Pa
- $R = 287{,}05$ J kg^{-1} K^{-1}

2.4. Rol dentro del modelo

Durante la integración temporal, el modelo dinámico consulta en cada paso:

- $LWC(z)$
- $w(z)$
- $T(z)$
- $\rho(z)$

Estos campos determinan el crecimiento por acreción, las fuerzas aerodinámicas y los intercambios térmicos del hidrometeoro.

2.5. Limitaciones

- Perfil estacionario (no dependiente del tiempo)
- Sin variabilidad horizontal
- No incluye turbulencia explícita
- Microfísica simplificada

3. Módulo: `hail_growth.py`

3.1. Propósito

Este módulo simula la trayectoria individual de un grano de granizo utilizando una integración numérica de cuarto orden (RK4) para las ecuaciones diferenciales que gobiernan la dinámica vertical.

Se incluyen efectos de:

- Crecimiento por acreción de agua líquida
- Fusión parcial a altas temperaturas
- Erosión por viento relativo
- Porosidad y compactación
- Efectos de sustentación aerodinámica
- Interacción con corrientes ascendentes de la nube

3.2. Función principal: `simulate_trajectory`

```
simulate_trajectory(profile: CloudProfile,
                    D0_mm,
                    z0,
                    T0_K,
                    v0=0.0,
                    t_max=3600.0,
                    dt=0.1,
                    ...)
```

Parámetros principales

Parámetro	Descripción
`profile`	Objeto `CloudProfile` con condiciones de la nube
`D0_mm`	Diámetro inicial del granizo (mm)
`z0`	Altura inicial (m)
`T0_K`	Temperatura inicial del grano (K)
`v0`	Velocidad inicial (m/s)
`t_max`	Tiempo máximo de simulación (s)
`dt`	Paso de integración (s)
`rho_init`	Densidad inicial del grano (kg/m^3)
`rho_ice`	Densidad del hielo (kg/m^3)
`rho_water`	Densidad del agua (kg/m^3)
`alpha_rim`	Coeficiente de compactación por riming
`v_scale`	Escala de velocidad para riming
`beta_melt`	Coeficiente de fusión térmica
`tau_base`	Constante de tiempo base para porosidad
`p_min,p_max`	Límites de porosidad
`C_d`	Coeficiente de arrastre aerodinámico
`g`	Aceleración de la gravedad (m/s^2)
`k_wind`	Factor de erosión por viento
`k_melt`	Factor de fusión por temperatura
`h_conv`	Coeficiente de transferencia convectiva (W/m^{2}K)
`c_h`	Capacidad calorífica del grano (J/kgK)
`tau_updraft`	Constante de tiempo para memoria de corriente ascenden

3.3. Modelo físico

1. **Cálculo de masa inicial:**

$$m_0 = \frac{4}{3}\pi \left(\frac{D_0}{2}\right)^3 \rho_{ice}$$

2. **Porosidad inicial:**

$$p_0 = \text{clip}\left(\frac{\rho_{water} - \rho_{init}}{\rho_{water} - \rho_{ice}}, p_{min}, p_{max}\right)$$

3. **Crecimiento y erosión:**

$$\mathrm{dm/dt}_{\mathrm{accretion}} = E_c\, A_{\mathrm{proj}}\, LWC\, |v_{rel}| \tag{8}$$

$$\mathrm{dm/dt}_{\mathrm{erosion}} = k_{wind}\, \rho_{air}\, A_{\mathrm{proj}}\, |v_{rel}|\, \mathrm{erosion_factor} \tag{9}$$

$$\mathrm{dm/dt}_{\mathrm{net}} = \mathrm{dm/dt}_{\mathrm{accretion}} - \mathrm{dm/dt}_{\mathrm{erosion}} \tag{10}$$

4. **Porosidad y compactación:**

$$dp/dt = -\alpha_{rim}\frac{dm_{riming}}{m}(1 + |v_{rel}|/v_{scale}) + \beta_{melt}\frac{|dm/dt_{net}|}{m}$$

5. **Balance térmico:**

$$dT/dt = \frac{Q_{\mathrm{conv}} + Q_{\mathrm{latent}}}{mc_h}, \quad Q_{\mathrm{conv}} = h_{conv}A_{surf}(T_{air} - T_{grain}), \quad Q_{\mathrm{latent}} = L_f\, dm/dt_{accretion}$$

6. **Drag y lift:** Re calculado dinámicamente, coeficiente de arrastre ajustado según Reynolds:

$$F_{drag} = 0{,}5\, C_d\, \rho_{air}\, A_{proj}\, v_{rel}|v_{rel}|, \quad a_{lift} = \frac{C_{lift}\rho_{air}A_{proj}w_{air}^2}{m}$$

7. **Movimiento vertical:**

$$dv/dt = -g + \mathrm{efecto_updraft} + a_{drag} + a_{lift}, \quad dz/dt = v$$

8. **Densidad progresiva:**

$$d\rho/dt = 0{,}3(\rho_{target} - \rho)$$

3.4. Integración numérica

Se emplea **RK4 clásico** para resolver el sistema de ODEs:

$$\mathrm{state}_{n+1} = \mathrm{state}_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{11}$$

con k_i calculados a partir de la función `derivatives`.

3.5. Registro de resultados

Se almacenan vectores temporales de:

- Tiempo
- Masa
- Radio
- Altura
- Velocidad
- Temperatura del grano
- Densidad efectiva
- Porosidad
- Velocidad terminal instantánea

Estos arrays se devuelven en un diccionario final.

3.6. Consideraciones técnicas

- Suspensión suave de partículas muy pequeñas para evitar caída excesiva.
- Seguridad física: masa, porosidad y densidad siempre positivas.
- Integración eficiente: todos los cálculos vectorizados usando NumPy.
- Memoria de updraft: factor de sustentación dependiente de exposición previa.

4. Módulo: `hail_growth_stochastic.py`

4.1. Propósito

Este módulo implementa simulaciones Monte Carlo del modelo de granizo HAILCAST 1D, generando múltiples partículas con condiciones iniciales aleatorias coherentes con el modelo determinista.

Permite:

- Generar condiciones iniciales de manera aleatoria
- Ejecutar trayectorias individuales usando la misma dinámica que el caso determinista
- Calcular la velocidad terminal teórica según Beard
- Monitorear el progreso de la simulación y tiempos por partícula

4.2. Funciones principales

4.2.1. `generate_initial_conditions_mc`

```
generate_initial_conditions_mc(
    n_particles,
    D0_mean=5.0,
    D0_std=2.0,
    z_birth_min=4000.0,
    z_birth_max=6000.0,
    T0_list=(263.15, 265.15),
    seed=None
)
```

Descripción Genera condiciones iniciales aleatorias para el ensamble Monte Carlo:

- `D0`: diámetro inicial (mm), normal limitado a $\geq 0{,}1$ mm
- `z0`: altura de nacimiento (m), uniforme entre `z_birth_min` y `z_birth_max`
- `T0`: temperatura inicial del grano (K), seleccionada de `T0_list`

Salida Tres vectores: `D0`, `z0`, `T0`.

4.2.2. `run_montecarlo`

```
run_montecarlo(
    profile,
    n_particles=100,
    seed=None,
    progress_every=10,
    warn_time=5.0,
    **trajectory_kwargs
)
```

Descripción Ejecuta un ensamble Monte Carlo de `n_particles` partículas:

- Para cada grano, se genera una trayectoria usando `simulate_trajectory`.
- Se calcula la velocidad terminal teórica de Beard (`beard_terminal_velocity`).
- Se agrega metadata: altura de nacimiento (`z_birth`) y temperatura inicial (`T0_init`).
- Monitoreo de progreso cada `progress_every` partículas.
- Genera advertencias si una partícula tarda más de `warn_time` segundos en simularse.

Salida Lista de diccionarios, cada uno con:

- Datos de trayectoria (`time`, `height`, `velocity`, etc.)
- `D_final_mm`: diámetro final del grano (mm)
- `v_terminal_beard`: velocidad terminal según Beard (m/s)
- `z_birth`, `T0_init`: metadata inicial de la partícula

4.3. Consideraciones técnicas

- Uso de `numpy.random.Generator` para reproducibilidad con `seed`.
- Control de progreso para ensambles grandes, útil en experimentos con cientos de partículas.
- Integración directa con el módulo determinista, asegurando consistencia física.
- La función soporta parámetros adicionales de la trayectoria a través de `**trajectory_kwargs`.
- La velocidad terminal de Beard proporciona referencia física adicional para comparación con la dinámica simulada.

5. Módulo: `hailcast1d.py`

5.1. Propósito

Este módulo centraliza la ejecución de simulaciones de granizo para HAILCAST 1D. Integra:

- Modelo de nube (`CloudProfile`)
- Dinámica de granizo (`simulate_trajectory`)
- Simulaciones deterministas y Monte Carlo
- Organización de casos por tipo de corriente ascendente (*updraft*)

Proporciona una API de alto nivel para que el usuario pueda generar simulaciones individuales, conjuntos de partículas o experimentos completos sin preocuparse por la integración interna de las ODEs.

5.2. Funciones principales

5.2.1. 1. `run_deterministic_simulation`

```
run_deterministic_simulation(profile, D0_mm, T0_K, z0, rho_init,
    **kwargs)
```

Descripción Simula la trayectoria de un solo grano de granizo utilizando la dinámica realista definida en `simulate_trajectory`. Devuelve todas las variables, incluyendo velocidad terminal.

Salida Diccionario con:

- `diameter`: diámetro (mm)
- `time`: vector temporal (s)
- `height`: altura (m)
- `velocity`: velocidad (m/s)
- `v_terminal`: velocidad terminal instantánea (m/s)
- `mass`: masa (kg)
- `temperature`: temperatura del grano (K)
- `density`: densidad efectiva (kg/m^3)
- `D0_init`: diámetro inicial

5.2.2. 2. `run_montecarlo_simulations`

```
run_montecarlo_simulations(profile, D0_mean, D0_std, n_particles,
    z0, rho_init, seed=None, **kwargs)
```

Descripción Ejecuta *n_particles* trayectorias independientes con diámetros iniciales distribuidos normalmente (`D0_mean`, `D0_std`). Permite analizar distribuciones de granizo para una nube dada.

Salida Lista de diccionarios con las mismas claves que `run_deterministic_simulation` para cada grano.

5.2.3. 3. `run_case`

```
run_case(profile, D0_mm, T0_K, z0, rho_init, **kwargs)
```

Descripción Simulación de un solo caso con parámetros específicos de diámetro, temperatura y altura. Devuelve todas las variables de trayectoria para un grano individual.

5.2.4. 4. `run_updraft_set`

```
run_updraft_set(updraft_type, D0_list, T0_list, z0, rho_init,
   profile=None, **kwargs)
```

Descripción Ejecuta múltiples simulaciones deterministas para una lista de diámetros (`D0_list`) y temperaturas (`T0_list`) para un tipo de *updraft* especificado (`strong`, `dry`, `weak`).

Salida Diccionario con resultados indexados por combinación de diámetro y temperatura.

5.2.5. 5. `run_all_cases`

```
run_all_cases(D0_list, T0_list, z0, rho_init, **kwargs)
```

Descripción Ejecuta todos los casos para los tres tipos de corrientes ascendentes (`strong`, `dry`, `weak`). Devuelve un diccionario jerárquico:

- Clave principal: tipo de *updraft*
- Subclave: combinación de diámetro y temperatura
- Valor: resultados individuales por grano

5.3. Consideraciones técnicas

- Todas las simulaciones internas usan `simulate_trajectory` para garantizar consistencia física y velocidad terminal realista.
- Integración de parámetros adicionales vía `**kwargs`.
- Monte Carlo permite análisis estadístico de distribuciones de granizo.
- Flexibilidad para definir un perfil de nube propio o usar valores predeterminados.

6. Módulo: `plot_hail_trajectories.py`

6.1. Propósito

Este módulo provee funciones de visualizacion para HAILCAST 1D:

- Graficar trayectorias de granizo individuales (deterministas)
- Superponer trayectorias Monte Carlo para visualizar dispersion
- Mostrar histogramas de tamaños iniciales en ensambles Monte Carlo
- Control de estetica y guardado de graficos

6.2. Funciones principales

6.2.1. `plot_hailstone_trajectories`

```
plot_hailstone_trajectories(
    determinista_results,
    montecarlo_results=None,
    title='Trayectorias de granizo',
    save_path=None
)
```

Parametros

- `determinista_results`: diccionario con resultados deterministas
- `montecarlo_results`: lista de diccionarios con resultados Monte Carlo (opcional)
- `title`: titulo del grafico principal
- `save_path`: ruta para guardar el grafico (opcional)

Descripcion

1. Grafica la altura vs. tiempo para cada grano determinista:
    - Lineas con distintos colores y marcadores
    - Marcadores huecos para claridad visual
    - Leyenda con diametro inicial de cada grano
2. Si se proveen resultados Monte Carlo:
    - Superpone trayectorias individuales en gris con transparencia
    - Genera un histograma de los diametros iniciales
    - Marca media y desviacion estandar
3. Permite guardar el grafico principal y el histograma en archivos separados

Salida

- Graficos interactivos usando Matplotlib
- Archivos PNG si `save_path` es proporcionado

6.3. Consideraciones tecnicas

- Uso de `plt.cm.tab10` para paleta de colores consistente
- Control de `markevery` para no saturar el grafico con marcadores
- Histogramas con lineas de media y desviacion estandar para analisis estadistico
- Compatible con ensambles de cualquier tamaño
- Codigo preparado para integrar visualizacion de trayectorias deterministas y Monte Carlo de manera clara y comparativa